

Validation Activities

Running the software at every level — from a single unit, up to the whole system the customer signs off.

Unit · Integration · Functional · System · Acceptance

WHY THIS TOPIC EXISTS

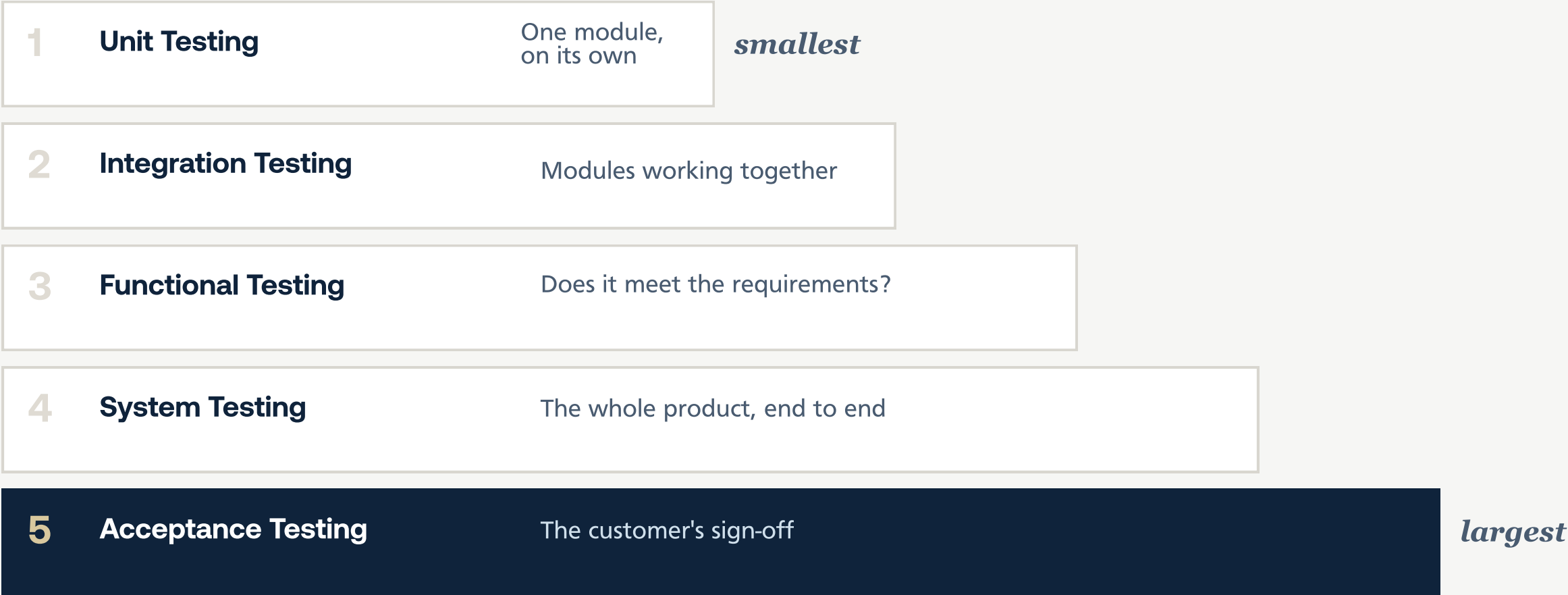
We start by ‘testing-in-the-small’ and work our way up to ‘testing-in-the-large.’

Validation asks the big question: are we building the RIGHT product — one that actually works and meets the customer's needs? You can't answer that with a single test. A large system is built from small parts, so we validate in stages: first each part alone, then the parts together, then the whole system, and finally the customer's own sign-off. Each level catches faults the level below it could not.

Unlike static testing, every validation activity here RUNS the software and watches what it does.

Five Levels of Validation

Each level widens the lens — from one module to the finished product in the customer's hands.



Unit Testing

In unit testing, individual modules or units are tested on their own. The purpose is to check that each piece of code performs the exact functionality it was written for. It is performed by the developers — and because they can see the code, it is a form of white-box testing, using those techniques to ensure coverage.

Small example:

Before assembling a car, you test the single spark plug by itself — not the whole engine. Unit testing checks one 'login()' function or one 'calculateTax()' method in isolation.

UNIT TEST CONSIDERATIONS

- The data structure that stores test data keeps its value through the whole program.
- Control statements are checked — do the boundary conditions restrict the test data correctly?
- All paths in the source code should be tested.

Integration Testing

Once the units work alone, we test them together as a group. The focus shifts to the COMMUNICATION between modules — checking for bugs in the way they interact. This matters because modules are written by different programmers, so their logic and coding style may differ; testing them together as a single entity is essential.

TWO HELPERS YOU'LL MEET

STUB

A dummy 'called' program. Stands in for a LOWER-level module that isn't ready yet, so a higher module can be tested now.

DRIVER

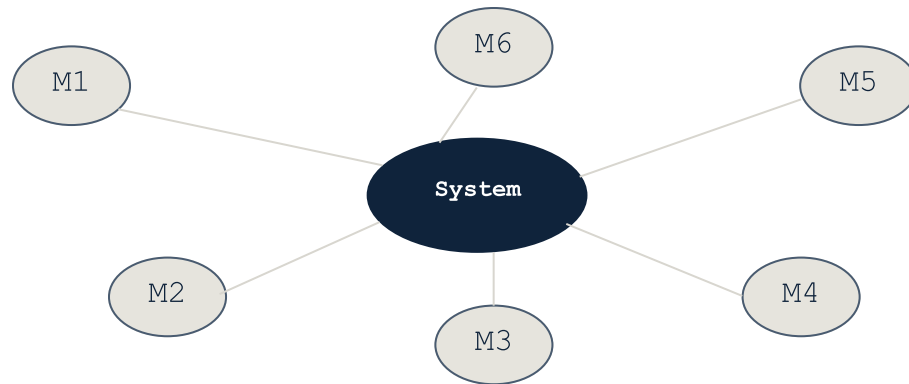
A dummy 'calling' program. Stands in for a HIGHER-level module that isn't ready, calling the module we want to test.

Four approaches follow: Big Bang, Top-down, Bottom-up, and Sandwich.

Big Bang & Top-Down

1 · BIG BANG

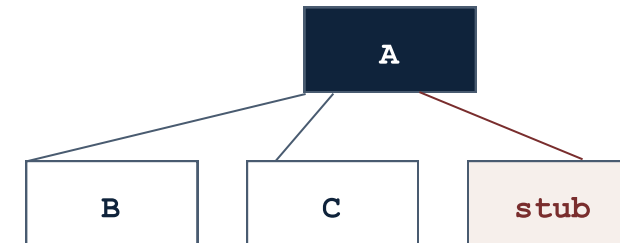
All modules are combined at once and tested as a single unit — only after every module is ready.



Convenient for small systems — but fault localisation is hard, and interface links are easily missed.

2 · TOP-DOWN

Higher-level modules are tested first; missing lower modules are replaced by STUBS, then swapped for real ones one by one.

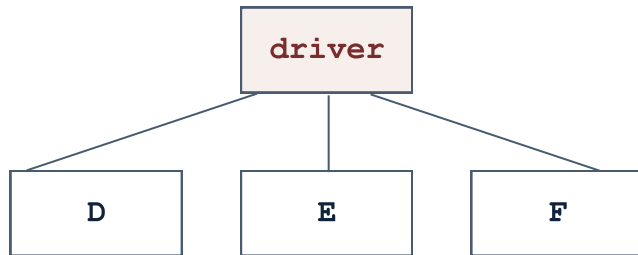


Easier to locate faults, and important modules can be tested first. Regression testing keeps new errors out.

Bottom-Up & Sandwich

3 · BOTTOM-UP

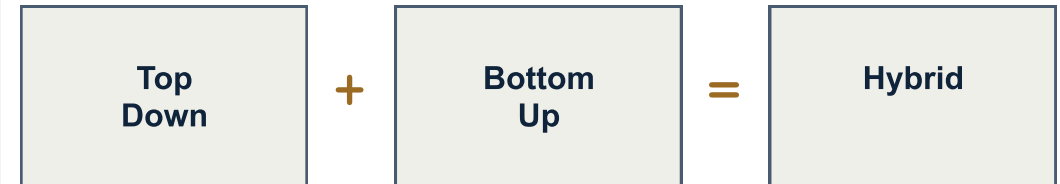
Lower-level modules are tested first, then used to test higher ones — upward until the top is reached. Missing higher modules are replaced by DRIVERS.



Lower modules are grouped into 'builds' and integrated. Good when the core logic lives in the lower layers.

4 · SANDWICH (HYBRID)

Both lower and higher modules are tested at the same time, then combined — a mix of top-down and bottom-up, using both drivers AND stubs.



Gives you the advantages of both approaches at once.

Functional Testing

Functional testing checks whether the software conforms to the requirement specifications — by entering inputs and seeing whether the output is exactly what was expected. It is a black-box activity: the internal code doesn't matter here, only the behaviour.

Small example:

Type a correct username and password → you should reach the dashboard. Type a wrong one → you should see an error. If it behaves that way, the login FUNCTION passes.

TWO WAYS TO DO IT

Based on Requirements

Testing is performed to check that all the stated requirements are met.

Based on Business Scenarios

Testing checks that all the business objectives of the software are met.

System Testing

System testing is conducted on the whole, integrated system to estimate its compliance with the specified set of requirements. It checks how the system behaves as a complete unit — validating both its functionality and its performance.

Whole system

The complete, integrated product — not individual parts.

Tester only

The user is NOT involved at this stage.

After integration

Comes after integration testing, before acceptance testing.

Both kinds

Involves both functional AND non-functional testing.

Types of System Testing

Because it looks at the whole product, system testing is really a family of checks — each guarding a different quality.

Functionality

Does it work as expected?

Recoverability

Can it recover from failures?

Interoperability

Can it work with other software?

Performance

Is it fast, stable, responsive?

Scalability

Can it grow to handle more?

Security

Is unauthorised access blocked?

Usability

Is it easy to learn and use?

Installation

Does it install correctly?

Functional vs. Non-Functional

System testing covers both. It's worth knowing the difference — it comes up again and again.

FUNCTIONAL

What the system DOES

The features and behaviour — checked against the requirements.

EXAMPLES

- 'Add to cart' adds the item
- Correct login opens the dashboard
- The invoice total is calculated right

NON-FUNCTIONAL

How WELL it does it

The qualities — speed, security, usability, reliability.

EXAMPLES

- The page loads within 2 seconds
- It supports 10,000 users at once
- Passwords are stored securely

Acceptance Testing

The final gate: the software is tested to ensure it will be accepted by the users, and complies with the business requirements. The user is completely involved, and it happens AFTER system testing. If the acceptance tests don't pass, the customer can refuse to take delivery of the product.

It determines whether the customer is satisfied — and focuses on functional testing based on the requirements the client gave.

ALPHA TESTING

Performed at the DEVELOPER'S site.

The team invites users in-house to try the product on home ground, in a controlled setting.

BETA TESTING

Performed by the USER (may not be at the developer's site).

Real users try it in the real world — catching issues the lab never could.

UAT & BAT

USER ACCEPTANCE TESTING

“Does it meet the user's needs?”

Conducted by the customer to ensure the system satisfies the contractual acceptance criteria before it is signed off as meeting user needs. The planning and execution need not be done by the customer directly — third-party consulting firms often do it — but the customer must specify the acceptance criteria to look for.

BUSINESS ACCEPTANCE TESTING

“A rehearsal of the UAT.”

Undertaken within the supplier's own development organisation to ensure the system will eventually PASS the UAT. It is a rehearsal of the UAT, held at the supplier's premises: the supplier derives and executes test cases from the client's contract — which includes the acceptance criteria.

The Importance of Testing in Levels

1 **Catch faults where they're cheapest**

A bug found in unit testing costs a fraction of the same bug found by a customer. Each level is a safety net beneath the next.

2 **Pinpoint the problem fast**

When you test small parts first, a failure points straight to the guilty module — far quicker than debugging a whole system at once.

3 **Build confidence step by step**

Passing each level — unit, integration, system — gives measurable, growing assurance that the product is ready.

4 **End with the only verdict that counts**

Acceptance testing puts the real user in charge: the product isn't 'done' until the customer agrees it meets their needs.